

C.N. PRACTICAL EXAM KEY ANSWERS

BATCH-2

- 1.) **Write a program to design a TCP Client-Server application to transfer a file.**

Ans:- AIM

To design and implement a TCP Client-Server application in Java to transfer a file from server to client.

ALGORITHM

Server Side:

1. Start the server.
2. Create a ServerSocket on a specific port.
3. Wait for client connection using accept().
4. Open the file to be sent.
5. Read file contents using FileInputStream.
6. Send data to client using OutputStream.

7. Close file and connection.

Client Side:

1. Start the client.
2. Connect to server using IP and port.
3. Create a file to store received data.
4. Read data from server using InputStream.
5. Write data into file using FileOutputStream.
6. Close connection.

PROGRAM

SERVER:-

```

import java.io.*;
import java.net.*;

public class FileServer {
    public static void main(String[] args) {
        try {
            ServerSocket serverSocket = new ServerSocket(5000);
            System.out.println("Server is waiting for connection...");

            Socket socket = serverSocket.accept();
            System.out.println("Client connected!");

            File file = new File("test.txt");
            FileInputStream fis = new FileInputStream(file);
            OutputStream os = socket.getOutputStream();

            byte[] buffer = new byte[1024];
            int bytesRead;

            while ((bytesRead = fis.read(buffer)) > 0) {
                os.write(buffer, 0, bytesRead);
            }

            System.out.println("File sent successfully!");

            fis.close();
            os.close();
            socket.close();
            serverSocket.close();

        } catch (Exception e) {
            System.out.println(e);
        }
    }
}

```

CLIENT:-

```

import java.io.*;
import java.net.*;

public class FileClient {
    public static void main(String[] args) {
        try {
            Socket socket = new Socket("localhost", 5000);
            System.out.println("Connected to server!");

            InputStream is = socket.getInputStream();
            FileOutputStream fos = new FileOutputStream("received.txt");

            byte[] buffer = new byte[1024];
            int bytesRead;

            while ((bytesRead = is.read(buffer)) > 0) {
                fos.write(buffer, 0, bytesRead);
            }

            System.out.println("File received successfully!");

            fos.close();
            is.close();
            socket.close();

        } catch (Exception e) {
            System.out.println(e);
        }
    }
}

```

OUTPUT:-

SERVER:-

```

Server is waiting for connection...
Client connected!
File sent successfully!

```

CLIENT:-

```

Connected to server!
File received successfully!

```

- 2.) Using TCP socket download a webpage and get the URL and pass it for buffering the content and write it as a html file.

Ans:- **AIM**

To write a Java program using TCP sockets to download a webpage by passing a URL, buffer its content, and store it as an HTML file.

ALGORITHM

1. Start the program
2. Accept URL (host + path)
3. Create a socket connection to the web server (port 80)
4. Send HTTP GET request
5. Read response using buffering (BufferedReader)
6. Store response content into an .html file
7. Close connection
8. Stop

PROGRAM

```

import java.io.*;
import java.net.*;

public class WebPageDownloader {
    public static void main(String[] args) {
        try {
            String host = "example.com"; // website
            String path = "/";           // page path

            // create socket (TCP)
            Socket socket = new Socket(host, 80);

            // send HTTP request
            PrintWriter pw = new PrintWriter(socket.getOutputStream(), true);
            pw.println("GET " + path + " HTTP/1.1");
            pw.println("Host: " + host);
            pw.println("Connection: close");
            pw.println(); // blank line

            // read response
            BufferedReader br = new BufferedReader(
                new InputStreamReader(socket.getInputStream()));

            // write to html file
            BufferedWriter bw = new BufferedWriter(
                new FileWriter("output.html"));

            String line;
            while ((line = br.readLine()) != null) {
                bw.write(line);
                bw.newLine();
            }

            System.out.println("Webpage downloaded successfully!");

```

```

                // close resources
                bw.close();
                br.close();
                pw.close();
                socket.close();

            } catch (Exception e) {
                e.printStackTrace();
            }
        }
    }
}

```

OUTPUT

```

C:\Users\HP\Downloads\Computer Networks
Webpage downloaded successfully!

```

3.) Write a program to implement the TCP Client Server Chat Application using Sockets.

Ans:- **AIM**

To implement a TCP Client-Server Chat Application using Java Sockets for two-way communication.

ALGORITHM

Server:

1. Create ServerSocket on a port
2. Wait for client connection
3. Create input/output streams
4. Continuously:
 - Read message from client
 - Display it
 - Send reply to client
5. Repeat until chat ends

Client:

1. Connect to server using IP and port
2. Create input/output streams

3. Continuously:

- Send message to server
- Receive reply
- Display it

4. Repeat until chat ends

PROGRAM

SERVER:-


```

import java.io.*;
import java.net.*;

public class ChatServer {
    public static void main(String[] args) {
        try {
            ServerSocket ss = new ServerSocket(5000);
            System.out.println("Server waiting...");

            Socket s = ss.accept();
            System.out.println("Client connected!");

            DataInputStream dis = new DataInputStream(s.getInputStream());
            DataOutputStream dos = new DataOutputStream(s.getOutputStream());

            BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

            String msg = "", reply = "";

            while (!msg.equals("exit")) {
                msg = dis.readUTF();
                System.out.println("Client: " + msg);

                System.out.print("Server: ");
                reply = br.readLine();
                dos.writeUTF(reply);
            }

            dis.close();
            dos.close();
            s.close();
            ss.close();

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

CLIENT:-

```

import java.io.*;
import java.net.*;

public class ChatClient {
    public static void main(String[] args) {
        try {
            Socket s = new Socket("localhost", 5000);
            System.out.println("Connected to server!");

            DataInputStream dis = new DataInputStream(s.getInputStream());
            DataOutputStream dos = new DataOutputStream(s.getOutputStream());

            BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

            String msg = "", reply = "";

            while (!msg.equals("exit")) {
                System.out.print("Client: ");
                msg = br.readLine();
                dos.writeUTF(msg);

                reply = dis.readUTF();
                System.out.println("Server: " + reply);
            }

            dis.close();
            dos.close();
            s.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

OUTPUT:-

SERVER:-

```

Server waiting...
Client connected!
Client: Hello
Server: Hi
Client: How are you
Server: I am fine , how are you ?
Client: Im good too
Server: |

```

CLIENT:-

```
Connected to server!  
Client: Hello  
Server: Hi  
Client: How are you  
Server: I am fine , how are you ?  
Client: Im good too  
|
```

- 4.) Write a programs hello_server for TCP (The client connects to the server, sends the string “Hello, world!”, then closes the connection).

Ans:- **AIM**

To implement a TCP client-server program where the client sends the message “**Hello, world!**” to the server and then closes the connection.

ALGORITHM

Server:

1. Create ServerSocket on a port
2. Wait for client connection
3. Accept connection
4. Read message from client

5. Display the message

6. Close connection

Client:

1. Connect to server using IP and port

2. Send the message “Hello, world!”

3. Close connection

PROGRAM:-

SERVER:-

```
import java.io.*;
import java.net.*;

public class HelloServer {
    public static void main(String[] args) {
        try {
            ServerSocket ss = new ServerSocket(5000);
            System.out.println("Server waiting...");

            Socket s = ss.accept();
            System.out.println("Client connected");

            DataInputStream dis = new DataInputStream(s.getInputStream());

            String message = dis.readUTF();
            System.out.println("Message from client: " + message);

            dis.close();
            s.close();
            ss.close();

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

CLIENT:-

```
import java.io.*;
import java.net.*;

public class HelloClient {
    public static void main(String[] args) {
        try {
            Socket s = new Socket("localhost", 5000);

            DataOutputStream dos = new DataOutputStream(s.getOutputStream());

            dos.writeUTF("Hello, world!");

            dos.close();
            s.close();

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

OUTPUT:-

SERVER:-

```
Server waiting...
Client connected
Message from client: Hello, world!
```

CLIENT:-

No Output – Message Sent Successfully!

- 5.) Write the program for Simulation of DNS using UDP Socket by getting the frame size from the

user and create the frame sized user request and send the frame to server from the client.

Ans:- AIM

To simulate DNS using UDP sockets by taking frame size from the user, creating a frame-sized request at the client, and sending it to the server.

ALGORITHM

Server:

1. Create a DatagramSocket on a port
2. Wait to receive packet from client
3. Extract domain name from received frame
4. Simulate DNS lookup (map domain → IP)
5. Send response back to client
6. Close socket

Client:

1. Get frame size from user
2. Get domain name from user
3. Pad the request to match frame size
4. Create DatagramSocket

5. Send request packet to server
6. Receive response from server
7. Display IP address
8. Close socket

PROGRAM

SERVER:-

```

import java.net.*;

public class DNSServer {
    public static void main(String[] args) {
        try {
            DatagramSocket ds = new DatagramSocket(5000);
            System.out.println("DNS Server is running...");

            byte[] receive = new byte[1024];

            DatagramPacket dp = new DatagramPacket(receive, receive.length);
            ds.receive(dp);

            String request = new String(dp.getData()).trim();
            System.out.println("Received domain: " + request);

            // Simulated DNS mapping
            String response;
            if (request.equals("google.com"))
                response = "142.250.190.78";
            else if (request.equals("yahoo.com"))
                response = "98.137.11.163";
            else
                response = "Domain not found";

            byte[] send = response.getBytes();

            DatagramPacket dpSend = new DatagramPacket(
                send, send.length,
                dp.getAddress(), dp.getPort());

            ds.send(dpSend);

            System.out.println("Response sent");

            ds.close();

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

CLIENT:-


```

import java.net.*;
import java.util.Scanner;

public class DNSClient {
    public static void main(String[] args) {
        try {
            Scanner sc = new Scanner(System.in);

            System.out.print("Enter frame size: ");
            int frameSize = sc.nextInt();
            sc.nextLine();

            System.out.print("Enter domain name: ");
            String domain = sc.nextLine();

            // pad to frame size
            StringBuilder frame = new StringBuilder(domain);
            while (frame.length() < frameSize) {
                frame.append(" "); // padding
            }

            byte[] send = frame.toString().getBytes();

            DatagramSocket ds = new DatagramSocket();

            InetAddress ip = InetAddress.getByName("localhost");

            DatagramPacket dp = new DatagramPacket(send, send.length, ip, 5000);
            ds.send(dp);

            byte[] receive = new byte[1024];
            DatagramPacket dpReceive = new DatagramPacket(receive, receive.length);

            ds.receive(dpReceive);

            String response = new String(dpReceive.getData()).trim();

            System.out.println("IP Address: " + response);

            ds.close();
            sc.close();

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

OUTPUT:-

SERVER:-

```

DNS Server is running...
Received domain: google.com
Response sent

```

CLIENT:-

```
Enter frame size: 20
Enter domain name: google.com
IP Address: 142.250.190.78
```

- 6.) Write a program to implement to transfer the files using TCP socket. The server send a reply to the user with the files. The user specified file needs to be downloaded.

Ans:- **AIM**

To implement a TCP client-server program where the client requests a specific file and the server sends the requested file to the client.

ALGORITHM

Server:

1. Start server on a port
2. Wait for client connection
3. Receive requested file name from client
4. Check if file exists
5. If exists:

- Send file size
- Send file data

6. Else:

- Send error message

7. Close connection

Client:

1. Connect to server
2. Enter file name
3. Send file name to server
4. Receive response
5. If file exists:
 - Receive file size
 - Receive file data and save
6. Else display error
7. Close connection

PROGRAM:-

SERVER:-

```

import java.io.*;
import java.net.*;

public class FileServer {
    public static void main(String[] args) {
        try {
            ServerSocket ss = new ServerSocket(5000);
            System.out.println("Server waiting...");

            Socket s = ss.accept();
            System.out.println("Client connected");

            DataInputStream dis = new DataInputStream(s.getInputStream());
            DataOutputStream dos = new DataOutputStream(s.getOutputStream());

            // receive filename
            String fileName = dis.readUTF();
            File file = new File(fileName);

            if (file.exists()) {
                dos.writeUTF("FOUND");
                dos.writeLong(file.length());

                FileInputStream fis = new FileInputStream(file);
                byte[] buffer = new byte[4096];
                int count;

                while ((count = fis.read(buffer)) != -1) {
                    dos.write(buffer, 0, count);
                }

                fis.close();
                System.out.println("File sent: " + fileName);
            } else {
                dos.writeUTF("NOTFOUND");
                System.out.println("File not found: " + fileName);
            }

            dos.flush();

            dis.close();
            dos.close();
            s.close();
            ss.close();

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

CLIENT:-

```

import java.io.*;
import java.net.*;
import java.util.Scanner;

public class FileClient {
    public static void main(String[] args) {
        try {
            Socket s = new Socket("localhost", 5000);
            System.out.println("Connected to server");

            DataInputStream dis = new DataInputStream(s.getInputStream());
            DataOutputStream dos = new DataOutputStream(s.getOutputStream());
            Scanner sc = new Scanner(System.in);

            System.out.print("Enter file name: ");
            String fileName = sc.nextLine();

            // send filename
            dos.writeUTF(fileName);

            String status = dis.readUTF();

            if (status.equals("FOUND")) {
                long fileSize = dis.readLong();

                FileOutputStream fos = new FileOutputStream("downloaded_" + fileName);

                byte[] buffer = new byte[4096];
                int count;
                long remaining = fileSize;

                while ((count = dis.read(buffer, 0, (int)Math.min(buffer.length, remaining))) > 0) {
                    fos.write(buffer, 0, count);
                    remaining -= count;
                    if (remaining == 0) break;
                }

                fos.close();
                System.out.println("File downloaded successfully");

            } else {
                System.out.println("File not found on server");
            }

            dis.close();
            dos.close();
            s.close();
            sc.close();

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

OUTPUT:-

SERVER:-

```
Server waiting...  
Client connected  
File sent: send.txt
```

CLIENT:-

```
Connected to server  
Enter file name: send.txt  
File downloaded successfully
```

7.) Using Wire shark capture the FTP user name and password.

Ans:- **AIM**

To capture FTP username and password using **Wireshark** by analyzing FTP packets.

ALGORITHM

1. Install and open **Wireshark**
2. Start capturing packets on the active network interface

3. Apply display filter:

ftp

4. Connect to an FTP server (use a **test server / your own setup**)

5. Login using username and password

6. Stop packet capture

7. Search for packets containing:

- USER
- PASS

8. Inspect packet details to view credentials

PROGRAM/PROCEDURE :-

PROCEDURE / STEPS

1. Open **Wireshark**

2. Select interface (Wi-Fi/Ethernet)

3. Click **Start Capture**

4. In another terminal, connect to FTP:

ftp localhost

5. Enter:

Username: test

Password: test123

6. Go back to Wireshark → Stop capture

7. In filter bar, type:

`ftp.request.command == "USER" ||`

`ftp.request.command == "PASS"`

8. Click packets → Expand:

File Transfer Protocol

OUTPUT

In Wireshark packet details:

Request: USER test

Request: PASS test123

Username and password are visible in plaintext

8.) Simulate the Reverse Address Resolution Protocol (RARP) using UDP.

Ans:- **AIM**

To simulate Reverse Address Resolution Protocol (RARP) using UDP sockets in Java.

ALGORITHM

Server:

1. Create DatagramSocket on a port
2. Wait for client request (MAC address)
3. Convert received data to string
4. Compare MAC with predefined table
5. Find corresponding IP address
6. Send IP back to client
7. Close socket

Client:

1. Create DatagramSocket
2. Enter MAC address
3. Send MAC to server
4. Receive response (IP address)
5. Display result
6. Close socket

PROGRAM:-

SERVER:-

```
import java.net.*;

public class RARPServer {
    public static void main(String[] args) {
        try {
            DatagramSocket ds = new DatagramSocket(5000);
            System.out.println("RARP Server is running...");

            byte[] receive = new byte[1024];
            DatagramPacket dp = new DatagramPacket(receive, receive.length);

            ds.receive(dp);

            String mac = new String(dp.getData()).trim();
            System.out.println("Received MAC: " + mac);

            String ip;

            // Simulated RARP table
            if (mac.equals("AA:BB:CC:DD:EE:01"))
                ip = "192.168.1.1";
            else if (mac.equals("AA:BB:CC:DD:EE:02"))
                ip = "192.168.1.2";
            else
                ip = "IP not found";

            byte[] send = ip.getBytes();

            DatagramPacket dpSend = new DatagramPacket(
                send, send.length,
                dp.getAddress(), dp.getPort());

            ds.send(dpSend);

            System.out.println("Response sent");

            ds.close();

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

CLIENT:-

```

import java.net.*;
import java.util.Scanner;

public class RARPCClient {
    public static void main(String[] args) {
        try {
            Scanner sc = new Scanner(System.in);

            DatagramSocket ds = new DatagramSocket();

            InetAddress ip = InetAddress.getByName("localhost");

            System.out.print("Enter MAC Address: ");
            String mac = sc.nextLine();

            byte[] send = mac.getBytes();

            DatagramPacket dp = new DatagramPacket(send, send.length, ip, 5000);
            ds.send(dp);

            byte[] receive = new byte[1024];
            DatagramPacket dpReceive = new DatagramPacket(receive, receive.length);

            ds.receive(dpReceive);

            String response = new String(dpReceive.getData()).trim();

            System.out.println("IP Address: " + response);

            ds.close();
            sc.close();

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

OUTPUT:-

SERVER:-

```

RARP Server is running...
Received MAC: AA:BB:CC:DD:EE:01
Response sent

```

CLIENT:-

```
C:\Users\HP\Downloads\Computer Networks P1  
Enter MAC Address: AA:BB:CC:DD:EE:01  
IP Address: 192.168.1.1
```

- 9.) Write a program to find the least cost route between any two nodes using Distance Vector Routing algorithm.

Ans:- **AIM**

To find the least cost route between nodes using the Distance Vector Routing algorithm.

ALGORITHM

1. Input number of nodes
2. Input cost matrix (distance between nodes)
3. Initialize distance table for each node
4. For each node:
 - Compare its distance with all neighbors
 - Update shortest path using:

$$d[i][j] = \min(d[i][j], d[i][k] + d[k][j])$$

5. Repeat until no changes occur
6. Display shortest distance between all nodes

PROGRAM:-

```

import java.util.*;

public class DistanceVectorRouting {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of nodes: ");
        int n = sc.nextInt();

        int[][] cost = new int[n][n];

        System.out.println("Enter cost matrix (use 999 for infinity):");
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                cost[i][j] = sc.nextInt();
            }
        }

        int[][] dist = new int[n][n];

        // initialize distance table
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                dist[i][j] = cost[i][j];
            }
        }

        // Distance Vector Algorithm
        for (int k = 0; k < n; k++) {
            for (int i = 0; i < n; i++) {
                for (int j = 0; j < n; j++) {
                    if (dist[i][k] + dist[k][j] < dist[i][j]) {
                        dist[i][j] = dist[i][k] + dist[k][j];
                    }
                }
            }
        }

        System.out.println("\nShortest distance matrix:");
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                System.out.print(dist[i][j] + "\t");
            }
            System.out.println();
        }

        sc.close();
    }
}

```

OUTPUT:-

```

C:\Users\hp\Downloads\Computer Networks Practicals
Enter number of nodes: 3
Enter cost matrix (use 999 for infinity):
0 2 999
2 0 3
999 3 0

Shortest distance matrix:
0      2      5
2      0      3
5      3      0

```

10.) By using an any one Congestion Control mechanisms and simulate the functionalities using NS.

Ans:- **AIM**

To simulate TCP congestion control using **TCP Reno** in Network Simulator (NS2).

ALGORITHM

1. Create simulator object
2. Create nodes (source, router, destination)
3. Create duplex links between nodes
4. Set queue limits and delay
5. Attach TCP agent (Reno) to source

6. Attach sink agent to destination
7. Attach FTP application over TCP
8. Start and stop traffic at specific times
9. Run simulation and trace results

PROGRAM:-


```

import java.util.*;

public class LeakyBucket {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter bucket size: ");
        int bucketSize = sc.nextInt();

        System.out.print("Enter output rate: ");
        int outputRate = sc.nextInt();

        System.out.print("Enter number of packets: ");
        int n = sc.nextInt();

        int[] packets = new int[n];

        System.out.println("Enter packet sizes:");
        for (int i = 0; i < n; i++) {
            packets[i] = sc.nextInt();
        }

        int bucket = 0;

        System.out.println("\nTime\tIncoming\tBucket\tOutput\tDropped");

        for (int i = 0; i < n; i++) {
            int incoming = packets[i];
            int dropped = 0;

            if (bucket + incoming > bucketSize) {
                dropped = (bucket + incoming) - bucketSize;
                incoming -= dropped;
            }

            bucket += incoming;

            int output = Math.min(bucket, outputRate);
            bucket -= output;

            System.out.println((i+1) + "\t" + packets[i] + "\t\t" +
                               bucket + "\t" + output + "\t" + dropped);
        }

        sc.close();
    }
}

```

OUTPUT:-

```
Enter bucket size: 10
Enter output rate: 4
Enter number of packets: 5
Enter packet sizes:
5 7 3 6 4
```

Time	Incoming	Bucket	Output	Dropped
1	5	1	4	0
2	7	4	4	0
3	3	3	4	0
4	6	5	4	0
5	4	5	4	0

11.) Examine the through put performance of sliding window with varying packet sizes, error rates and Round Trip Times.

Ans:- **AIM**

To examine the throughput performance of Sliding Window protocol with varying packet sizes, error rates, and Round Trip Times (RTT).

ALGORITHM

1. Input:

- Packet size
- Window size
- RTT

- Error rate

2. Calculate transmission time

3. Compute successful transmission probability:

$$\text{success} = (1 - \text{errorRate})$$

4. Calculate throughput:

$$\text{throughput} = (\text{windowSize} * \text{packetSize} * \text{success}) / \text{RTT}$$

5. Repeat for different values

6. Display results

PROGRAM:-

```

import java.util.*;

public class SlidingWindowThroughput {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter window size: ");
        int windowSize = sc.nextInt();

        System.out.print("Enter number of test cases: ");
        int t = sc.nextInt();

        System.out.println("\nPktSize\tRTT\tErrorRate\tThroughput");

        for (int i = 0; i < t; i++) {
            System.out.println("\nTest Case " + (i + 1));

            System.out.print("Enter packet size (bits): ");
            double packetSize = sc.nextDouble();

            System.out.print("Enter RTT (seconds): ");
            double rtt = sc.nextDouble();

            System.out.print("Enter error rate (0 to 1): ");
            double errorRate = sc.nextDouble();

            double success = (1 - errorRate);

            double throughput = (windowSize * packetSize * success) / rtt;

            System.out.println(packetSize + "\t" + rtt + "\t" +
                               errorRate + "\t\t" + throughput + " bits/sec");
        }

        sc.close();
    }
}

```

OUTPUT:-

```

Enter window size: 4
Enter number of test cases: 2

PktSize RTT      ErrorRate      Throughput

Test Case 1
Enter packet size (bits): 1000
Enter RTT (seconds): 0.5
Enter error rate (0 to 1): 0.1
1000.0  0.5      0.1              7200.0 bits/sec

Test Case 2
Enter packet size (bits): 2000
Enter RTT (seconds): 1
Enter error rate (0 to 1): 0.2
2000.0  1.0      0.2              6400.0 bits/sec

```

12.) Compare the performance of a network by implementing Distance Vector Routing algorithm and link state routing.

Ans:- **AIM**

To compare the performance of a network using **Distance Vector Routing** and **Link State Routing** algorithms.

ALGORITHM

Distance Vector Routing (DVR)

1. Input number of nodes and cost matrix

2. Initialize distance table

3. Update distances using:

$$d[i][j] = \min(d[i][j], d[i][k] + d[k][j])$$

4. Repeat until stable

5. Display shortest paths

Link State Routing (Dijkstra)

1. Input number of nodes and cost matrix

2. Choose source node

3. Initialize distances

4. Select minimum distance node

5. Update neighbors

6. Repeat until all nodes visited

7. Display shortest path

PROGRAM:-

Distance Vector Routing:-

```

import java.util.*;

public class DVR {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of nodes: ");
        int n = sc.nextInt();

        int[][] dist = new int[n][n];

        System.out.println("Enter cost matrix (999 for infinity):");
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                dist[i][j] = sc.nextInt();
            }
        }

        for (int k = 0; k < n; k++) {
            for (int i = 0; i < n; i++) {
                for (int j = 0; j < n; j++) {
                    if (dist[i][k] + dist[k][j] < dist[i][j]) {
                        dist[i][j] = dist[i][k] + dist[k][j];
                    }
                }
            }
        }

        System.out.println("\nDVR Shortest Path:");
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                System.out.print(dist[i][j] + "\t");
            }
            System.out.println();
        }

        sc.close();
    }
}

```

Link State Routing (Dijkstra):-

```

import java.util.*;

public class LinkStateRouting {
    static int minDistance(int[] dist, boolean[] visited, int n) {
        int min = Integer.MAX_VALUE, index = -1;

        for (int i = 0; i < n; i++) {
            if (!visited[i] && dist[i] <= min) {
                min = dist[i];
                index = i;
            }
        }
        return index;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of nodes: ");
        int n = sc.nextInt();

        int[][] graph = new int[n][n];

        System.out.println("Enter cost matrix (0 if no edge):");
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                graph[i][j] = sc.nextInt();
            }
        }

        System.out.print("Enter source node: ");
        int src = sc.nextInt();

        int[] dist = new int[n];
        boolean[] visited = new boolean[n];

        Arrays.fill(dist, Integer.MAX_VALUE);
        dist[src] = 0;

        for (int count = 0; count < n - 1; count++) {
            int u = minDistance(dist, visited, n);
            visited[u] = true;

            for (int v = 0; v < n; v++) {
                if (!visited[v] && graph[u][v] != 0 &&
                    dist[u] + graph[u][v] < dist[v]) {
                    dist[v] = dist[u] + graph[u][v];
                }
            }
        }

        System.out.println("\nLink State (Dijkstra) Shortest Paths:");
        for (int i = 0; i < n; i++) {
            System.out.println("To node " + i + " = " + dist[i]);
        }

        sc.close();
    }
}

```

OUTPUT:-

DVR:-


```
Enter number of nodes: 3
Enter cost matrix (999 for infinity):
0 2 999
2 0 3
999 3 0

DVR Shortest Path:
0      2      5
2      0      3
5      3      0
```

LinkStateRouting:-

```
Enter number of nodes: 3
Enter cost matrix (0 if no edge):
0 2 0
2 0 3
0 3 0
Enter source node: 0

Link State (Dijkstra) Shortest Paths:
To node 0 = 0
To node 1 = 2
To node 2 = 5
```

13.) Implement the error correction techniques and examine the message send with error and send with no error.

Ans:- AIM

To implement error detection and correction using Hamming Code (even parity) and examine transmission with and without error.

ALGORITHM

Sender Side:

- 1. Accept 4-bit data**
- 2. Insert parity bits at positions 1, 2, 4**
- 3. Compute parity bits (even parity)**
- 4. Generate 7-bit Hamming code**
- 5. Send code**

Receiver Side:

- 1. Receive 7-bit code**
- 2. Recalculate parity bits**
- 3. Find error position (if any)**
- 4. If error exists → correct it**
- 5. Display corrected message**

PROGRAM:-

```

import java.util.*;

public class HammingCode {

    static int[] data = new int[7];

    static void calculateParityBits() {
        data[0] = data[2] ^ data[4] ^ data[6]; // p1
        data[1] = data[2] ^ data[5] ^ data[6]; // p2
        data[3] = data[4] ^ data[5] ^ data[6]; // p4
    }

    static int checkError() {
        int p1 = data[0] ^ data[2] ^ data[4] ^ data[6];
        int p2 = data[1] ^ data[2] ^ data[5] ^ data[6];
        int p4 = data[3] ^ data[4] ^ data[5] ^ data[6];

        return (p4 * 4 + p2 * 2 + p1);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("Enter 4-bit data:");
        int d1 = sc.nextInt();
        int d2 = sc.nextInt();
        int d3 = sc.nextInt();
        int d4 = sc.nextInt();

        // placing data bits
        data[2] = d1;
        data[4] = d2;
        data[5] = d3;
        data[6] = d4;

        calculateParityBits();

        System.out.println("\nTransmitted Code:");
        for (int i = 0; i < 7; i++) {
            System.out.print(data[i] + " ");
        }

        // simulate error
        System.out.println("\n\n--- Introducing Error at position 3 ---");
        data[2] = data[2] ^ 1; // flip bit

        System.out.println("Received Code:");
        for (int i = 0; i < 7; i++) {
            System.out.print(data[i] + " ");
        }

        int errorPos = checkError();

        if (errorPos == 0) {
            System.out.println("\nNo Error Detected");
        } else {
            System.out.println("\nError at position: " + errorPos);

```

```

            System.out.println("\nError at position: " + errorPos);

            // correct error
            data[errorPos - 1] ^= 1;

            System.out.println("Corrected Code:");
            for (int i = 0; i < 7; i++) {
                System.out.print(data[i] + " ");
            }

            sc.close();
        }
    }
}

```

OUTPUT:-

```
Enter 4-bit data:
1 0 1 1

Transmitted Code:
0 1 1 0 0 1 1

--- Introducing Error at position 3 ---
Received Code:
0 1 0 0 0 1 1
Error at position: 3
Corrected Code:
0 1 1 0 0 1 1
```

14.) Using network simulator implements the CRC error detection techniques between source and destination machine in network.

Ans:- AIM

To implement CRC (Cyclic Redundancy Check) error detection between source and destination using Network Simulator (NS2).

ALGORITHM

- 1. Create simulator object in NS2**
- 2. Create source and destination nodes**

- 3. Connect nodes using duplex link**
- 4. Attach TCP/UDP agents**
- 5. Attach application (FTP/CBR)**
- 6. Introduce error model (to simulate CRC check failure)**
- 7. Run simulation**
- 8. Analyze trace file for error detection**

PROGRAM:-

```

import java.util.*;

public class CRC {

    // XOR operation
    static String xor(String a, String b) {
        StringBuilder result = new StringBuilder();

        for (int i = 1; i < b.length(); i++) {
            result.append(a.charAt(i) == b.charAt(i) ? '0' : '1');
        }
        return result.toString();
    }

    // Division method
    static String divide(String dividend, String divisor) {
        int pick = divisor.length();

        String temp = dividend.substring(0, pick);

        while (pick < dividend.length()) {
            if (temp.charAt(0) == '1')
                temp = xor(divisor, temp) + dividend.charAt(pick);
            else
                temp = xor("0".repeat(pick), temp) + dividend.charAt(pick);

            pick++;
        }

        if (temp.charAt(0) == '1')
            temp = xor(divisor, temp);
        else
            temp = xor("0".repeat(pick), temp);

        return temp;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter data bits: ");
        String data = sc.nextLine();

        System.out.print("Enter generator: ");
        String gen = sc.nextLine();

        int genLen = gen.length();

        // Append zeros
        String appendedData = data + "0".repeat(genLen - 1);

        String remainder = divide(appendedData, gen);

        String codeword = data + remainder;

        System.out.println("\nTransmitted Codeword: " + codeword);
    }
}

```

```

        System.out.println("Transmitted Codeword: " + codeword);

        // ----- NO ERROR CASE -----
        String check = divide(codeword, gen);

        if (check.contains("1"))
            System.out.println("Error detected in transmission (NO ERROR CASE CHECK FAILED)");
        else
            System.out.println("No error detected (Correct transmission)");

        // ----- ERROR SIMULATION -----
        char[] errorData = codeword.toCharArray();
        errorData[2] = (errorData[2] == '0') ? '1' : '0'; // flip a bit

        String corrupted = new String(errorData);

        System.out.println("\nCorrupted Codeword: " + corrupted);

        String check2 = divide(corrupted, gen);

        if (check2.contains("1"))
            System.out.println("Error detected in corrupted transmission");
        else
            System.out.println("No error detected");

        sc.close();
    }
}

```

OUTPUT:-

```

Enter data bits: 1011
Enter generator: 1101

Transmitted Codeword: 1011100
No error detected (Correct transmission)

Corrupted Codeword: 1001100
Error detected in corrupted transmission

```

15.) Write a code simulating ARP protocol for the server and client.

Ans:- AIM

To simulate the Address Resolution Protocol (ARP) using Java TCP socket programming between a client and server.

ALGORITHM

Server:

- 1. Create ARP table (IP → MAC mapping)**
- 2. Start ServerSocket**
- 3. Wait for client request (IP address)**
- 4. Search IP in ARP table**
- 5. Send corresponding MAC address**
- 6. Close connection**

Client:

- 1. Connect to server**
- 2. Send IP address**
- 3. Receive MAC address**
- 4. Display result**
- 5. Close connection**

PROGRAM:-

SERVER:-


```

import java.io.*;
import java.net.*;
import java.util.*;

public class ARPServer {
    public static void main(String[] args) {
        try {
            ServerSocket ss = new ServerSocket(5000);
            System.out.println("ARP Server is running...");

            // ARP Table (IP -> MAC)
            Map<String, String> arpTable = new HashMap<>();
            arpTable.put("192.168.1.1", "AA:BB:CC:DD:EE:01");
            arpTable.put("192.168.1.2", "AA:BB:CC:DD:EE:02");
            arpTable.put("192.168.1.3", "AA:BB:CC:DD:EE:03");

            Socket s = ss.accept();
            System.out.println("Client connected");

            DataInputStream dis = new DataInputStream(s.getInputStream());
            DataOutputStream dos = new DataOutputStream(s.getOutputStream());

            String ip = dis.readUTF();
            System.out.println("Requested IP: " + ip);

            String mac = arpTable.getOrDefault(ip, "MAC NOT FOUND");

            dos.writeUTF(mac);

            dis.close();
            dos.close();
            s.close();
            ss.close();

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

CLIENT:-

```

import java.io.*;
import java.net.*;
import java.util.Scanner;

public class ARPClient {
    public static void main(String[] args) {
        try {
            Socket s = new Socket("localhost", 5000);
            System.out.println("Connected to ARP Server");

            DataInputStream dis = new DataInputStream(s.getInputStream());
            DataOutputStream dos = new DataOutputStream(s.getOutputStream());

            Scanner sc = new Scanner(System.in);

            System.out.print("Enter IP Address: ");
            String ip = sc.nextLine();

            dos.writeUTF(ip);

            String mac = dis.readUTF();

            System.out.println("MAC Address: " + mac);

            dis.close();
            dos.close();
            s.close();
            sc.close();

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

OUTPUT:-

SERVER:-

```

ARP Server is running...
Client connected
Requested IP: 192.168.1.2

```

CLIENT:-

```
Connected to ARP Server
Enter IP Address: 192.168.1.2
MAC Address: AA:BB:CC:DD:EE:02
```

16.) Write a program on data gram socket for client/server to display the messages on client side, typed at the server side.

Ans:- AIM

To implement a UDP Datagram Socket program where the server sends messages and the client displays them.

ALGORITHM

Server:

- 1. Create DatagramSocket**
- 2. Read message from user**
- 3. Convert message to byte array**
- 4. Send packet to client using IP + port**
- 5. Close socket**

Client:

- 1. Create DatagramSocket on port**
- 2. Wait to receive packet**
- 3. Extract message from packet**
- 4. Display message**
- 5. Close socket**

PROGRAM:-

SERVER:-

```

import java.net.*;
import java.util.Scanner;

public class UDPServer {
    public static void main(String[] args) {
        try {
            DatagramSocket ds = new DatagramSocket();

            Scanner sc = new Scanner(System.in);

            System.out.print("Enter message to send: ");
            String msg = sc.nextLine();

            byte[] sendData = msg.getBytes();

            InetAddress ip = InetAddress.getByName("localhost");

            DatagramPacket dp = new DatagramPacket(
                sendData,
                sendData.length,
                ip,
                5000
            );

            ds.send(dp);

            System.out.println("Message sent to client");

            ds.close();
            sc.close();

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

CLIENT:-

```

import java.net.*;

public class UDPClient {
    public static void main(String[] args) {
        try {
            DatagramSocket ds = new DatagramSocket(5000);

            byte[] receive = new byte[1024];

            DatagramPacket dp = new DatagramPacket(receive, receive.length);

            System.out.println("Client waiting for message...");

            ds.receive(dp);

            String msg = new String(dp.getData()).trim();

            System.out.println("Message from server: " + msg);

            ds.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

OUTPUT:-

SERVER:-

```

Enter message to send: Hello client
Message sent to client

```

CLIENT:-

```

Client waiting for message...
Message from server: Hello client

```

17.) Write the program for implementing the client for Simple mail transfer protocol.

Ans:-

AIM:-

To implement a client program for Simple Mail Transfer Protocol (SMTP) using Java socket programming.

ALGORITHM

- 1. Create a socket connection to SMTP server (port 25 or test server)**
- 2. Create input/output streams**
- 3. Send SMTP commands:**
 - HELO**
 - MAIL FROM**
 - RCPT TO**
 - DATA**
 - message body**
 - QUIT**

4. Read server responses

5. Close connection

PROGRAM:-

SERVER:-


```

import java.io.*;
import java.net.*;

public class SMTPServer {
    public static void main(String[] args) {
        try {
            ServerSocket ss = new ServerSocket(5000);
            System.out.println("SMTP Server started...");

            Socket s = ss.accept();
            System.out.println("Client connected");

            BufferedReader br = new BufferedReader(
                new InputStreamReader(s.getInputStream()));

            PrintWriter pw = new PrintWriter(s.getOutputStream(), true);

            pw.println("220 SMTP Server Ready");

            String msg;

            while ((msg = br.readLine()) != null) {
                System.out.println("Client: " + msg);

                if (msg.startsWith("HELO"))
                    pw.println("250 OK Hello");
                else if (msg.startsWith("MAIL FROM"))
                    pw.println("250 OK Sender");
                else if (msg.startsWith("RCPT TO"))
                    pw.println("250 OK Receiver");
                else if (msg.equals("DATA"))
                    pw.println("354 Start Input");
                else if (msg.equals("."))
                    pw.println("250 Message Received");
                else if (msg.equals("QUIT")) {
                    pw.println("221 Bye");
                    break;
                } else {
                    pw.println("250 OK");
                }
            }

            s.close();
            ss.close();

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

CLIENT:-

```

import java.io.*;
import java.net.*;
import java.util.Scanner;

public class SMTPClient {
    public static void main(String[] args) {
        try {
            Socket socket = new Socket("localhost", 5000);

            BufferedReader in = new BufferedReader(
                new InputStreamReader(socket.getInputStream()));

            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);

            Scanner sc = new Scanner(System.in);

            System.out.println(in.readLine());

            out.println("HELO localhost");
            System.out.println(in.readLine());

            System.out.print("Sender: ");
            String from = sc.nextLine();
            out.println("MAIL FROM:<" + from + ">");
            System.out.println(in.readLine());

            System.out.print("Receiver: ");
            String to = sc.nextLine();
            out.println("RCPT TO:<" + to + ">");
            System.out.println(in.readLine());

            out.println("DATA");
            System.out.println(in.readLine());

            System.out.println("Type message (end with .):");

            String msg;
            while (!(msg = sc.nextLine()).equals(".")) {
                out.println(msg);
            }

            out.println(".");
            System.out.println(in.readLine());

            out.println("QUIT");
            System.out.println(in.readLine());

            socket.close();
            sc.close();

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

OUTPUT:-

SERVER:-

```
SMTP Server started...
Client connected
Client: HELO localhost
Client: MAIL FROM:<user@gmail.com>
Client: RCPT TO:<test@gmail.com>
Client: DATA
Client: Hello this is test message
Client: .
Client: QUIT
```

CLIENT:-

```
220 SMTP Server Ready
250 OK Hello
Sender: user@gmail.com
250 OK Sender
Receiver: test@gmail.com
250 OK Receiver
354 Start Input
Type message (end with .):
Hello this is test message
.
250 OK
250 Message Received
```

18.) Write a code simulating RARP protocol for the server and client.

Ans:- To simulate Reverse Address Resolution Protocol (RARP) using Java TCP socket programming between client and server.

ALGORITHM

Server:

- 1. Create RARP table (MAC → IP mapping)**
- 2. Start ServerSocket**
- 3. Accept client connection**
- 4. Receive MAC address**
- 5. Search in table**
- 6. Send corresponding IP address**
- 7. Close connection**

Client:

- 1. Connect to server**
- 2. Send MAC address**
- 3. Receive IP address**
- 4. Display result**
- 5. Close connection**

PROGRAM

SERVER:-

```
import java.io.*;
import java.net.*;
import java.util.*;

public class RARPServer {
    public static void main(String[] args) {
        try {
            ServerSocket ss = new ServerSocket(5000);
            System.out.println("RARP Server started...");

            // MAC → IP table
            Map<String, String> rarpTable = new HashMap<>();
            rarpTable.put("AA:BB:CC:DD:EE:01", "192.168.1.1");
            rarpTable.put("AA:BB:CC:DD:EE:02", "192.168.1.2");
            rarpTable.put("AA:BB:CC:DD:EE:03", "192.168.1.3");

            Socket s = ss.accept();
            System.out.println("Client connected");

            BufferedReader br = new BufferedReader(
                new InputStreamReader(s.getInputStream()));

            PrintWriter pw = new PrintWriter(s.getOutputStream(), true);

            String mac = br.readLine();
            System.out.println("Received MAC: " + mac);

            String ip = rarpTable.getOrDefault(mac, "IP NOT FOUND");

            pw.println(ip);

            br.close();
            pw.close();
            s.close();
            ss.close();

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

CLIENT:-

```

import java.io.*;
import java.net.*;
import java.util.Scanner;

public class RARPCClient {
    public static void main(String[] args) {
        try {
            Socket s = new Socket("localhost", 5000);
            System.out.println("Connected to RARP Server");

            BufferedReader br = new BufferedReader(
                new InputStreamReader(s.getInputStream()));

            PrintWriter pw = new PrintWriter(s.getOutputStream(), true);

            Scanner sc = new Scanner(System.in);

            System.out.print("Enter MAC Address: ");
            String mac = sc.nextLine();

            pw.println(mac);

            String ip = br.readLine();

            System.out.println("IP Address: " + ip);

            br.close();
            pw.close();
            s.close();
            sc.close();

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

OUTPUT:-

SERVER:-

```

C:\Users\An (Computer) > java RARPServer
RARP Server started...
Client connected
Received MAC: AA:BB:CC:DD:EE:02

```

CLIENT:-

```
Connected to RARP Server
Enter MAC Address: AA:BB:CC:DD:EE:02
IP Address: 192.168.1.2
```

19.) a) Write a HTTP web client program to download any webpage using TCP sockets.

Ans:- AIM

To write a HTTP web client program using TCP sockets to download a webpage from a server.

ALGORITHM

- 1. Create a TCP socket connection to web server (port 80)**
- 2. Send HTTP GET request**
- 3. Read server response using input stream**
- 4. Store response in a file (.html)**
- 5. Close connection**

PROGRAM:-

```

import java.io.*;
import java.net.*;

public class HTTPClient {
    public static void main(String[] args) {
        try {
            String host = "example.com";    // website
            String path = "/";              // webpage path

            Socket socket = new Socket(host, 80);

            PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
            BufferedReader in = new BufferedReader(
                new InputStreamReader(socket.getInputStream()));

            // HTTP GET request
            out.println("GET " + path + " HTTP/1.1");
            out.println("Host: " + host);
            out.println("Connection: close");
            out.println();

            // write response to file
            BufferedWriter file = new BufferedWriter(
                new FileWriter("webpage.html"));

            String line;
            while ((line = in.readLine()) != null) {
                file.write(line);
                file.newLine();
            }

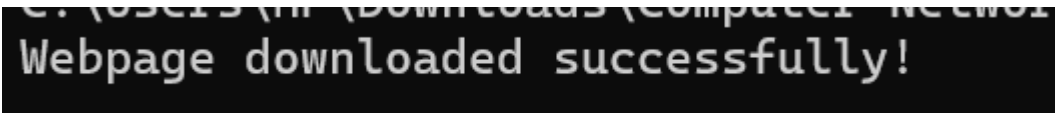
            System.out.println("Webpage downloaded successfully!");

            file.close();
            in.close();
            out.close();
            socket.close();

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

OUTPUT:-



```

C:\Users\Nir\Desktop\Computer Network
Webpage downloaded successfully!

```

20.) Use the network simulator to capture and investigate some packets exchanged at the network layer.

Ans:- AIM

To use Network Simulator (NS2) to capture and investigate packets exchanged at the network layer.

ALGORITHM

- 1. Create simulator object in NS2**
- 2. Create nodes (source, router, destination)**
- 3. Connect nodes using duplex links**
- 4. Enable trace file generation (.tr)**
- 5. Attach traffic agent (TCP/UDP + FTP/CBR)**
- 6. Run simulation**
- 7. Capture packets in trace file**
- 8. Analyze packet events (send, receive, drop)**

PROGRAM:-

```

import java.util.*;

class Packet {
    String src, dest, data;

    Packet(String src, String dest, String data) {
        this.src = src;
        this.dest = dest;
        this.data = data;
    }
}

class Network {
    Random rand = new Random();

    void sendPacket(Packet p) {
        System.out.println("s PACKET sent from " + p.src + " to " + p.dest);

        // simulate network condition
        int chance = rand.nextInt(100);

        if (chance < 70) { // 70% success
            System.out.println("r PACKET received at " + p.dest);
        } else { // 30% drop
            System.out.println("d PACKET dropped in network");
        }
    }
}

public class NetworkLayerSimulation {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Network net = new Network();

        System.out.print("Enter source: ");
        String src = sc.nextLine();

        System.out.print("Enter destination: ");
        String dest = sc.nextLine();

        System.out.print("Enter message: ");
        String msg = sc.nextLine();

        Packet p = new Packet(src, dest, msg);

        System.out.println("\n--- Network Simulation Start ---\n");

        for (int i = 0; i < 5; i++) {
            net.sendPacket(p);
        }

        System.out.println("\n--- Simulation End ---");

        sc.close();
    }
}

```

OUTPUT:-

```
Enter source: A
Enter destination: B
Enter message: Hello

--- Network Simulation Start ---

s PACKET sent from A to B
r PACKET received at B
s PACKET sent from A to B
d PACKET dropped in network
s PACKET sent from A to B
d PACKET dropped in network
s PACKET sent from A to B
r PACKET received at B
s PACKET sent from A to B
r PACKET received at B

--- Simulation End ---
```

21.)a) Using Simulation tool analyze the TCP/UDP performance.

Ans:- AIM

To analyze and compare the performance of TCP and UDP using Network Simulator (NS2).

ALGORITHM

- 1. Create simulator object**
- 2. Create nodes (source, router, destination)**
- 3. Establish duplex links**
- 4. Attach TCP flow (FTP traffic)**
- 5. Attach UDP flow (CBR traffic)**
- 6. Enable tracing (.tr file)**
- 7. Run simulation**
- 8. Analyze packet delivery, delay, and loss**

PROGRAM:-

```

import java.util.*;

public class TCPvsUDP {

    static Random rand = new Random();

    // TCP Simulation
    static void tcpSimulation(int packets) {
        System.out.println("\n--- TCP Simulation ---");

        int delivered = 0;

        for (int i = 1; i <= packets; i++) {
            try {
                Thread.sleep(100); // simulate delay
            } catch (Exception e) {}

            System.out.println("Packet " + i + " sent and ACK received (TCP)");
            delivered++;
        }

        System.out.println("TCP Delivered: " + delivered + "/" + packets);
    }

    // UDP Simulation
    static void udpSimulation(int packets) {
        System.out.println("\n--- UDP Simulation ---");

        int delivered = 0;
        int lost = 0;

        for (int i = 1; i <= packets; i++) {
            int chance = rand.nextInt(100);

            if (chance < 75) { // 75% success
                System.out.println("Packet " + i + " received (UDP)");
                delivered++;
            } else {
                System.out.println("Packet " + i + " lost (UDP)");
                lost++;
            }

            try {
                Thread.sleep(50); // faster than TCP
            } catch (Exception e) {}
        }

        System.out.println("UDP Delivered: " + delivered + "/" + packets);
        System.out.println("UDP Lost: " + lost);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of packets: ");
        int packets = sc.nextInt();

        tcpSimulation(packets);
        udpSimulation(packets);
    }
}

```

```

        udpSimulation(packets);

        System.out.println("\n--- PERFORMANCE COMPARISON ---");
        System.out.println("TCP: Reliable but slower");
        System.out.println("UDP: Faster but may lose packets");

        sc.close();
    }
}

```

OUTPUT:-

```
Enter number of packets: 5

--- TCP Simulation ---
Packet 1 sent and ACK received (TCP)
Packet 2 sent and ACK received (TCP)
Packet 3 sent and ACK received (TCP)
Packet 4 sent and ACK received (TCP)
Packet 5 sent and ACK received (TCP)
TCP Delivered: 5/5

--- UDP Simulation ---
Packet 1 lost (UDP)
Packet 2 received (UDP)
Packet 3 received (UDP)
Packet 4 received (UDP)
Packet 5 received (UDP)
UDP Delivered: 4/5
UDP Lost: 1

--- PERFORMANCE COMPARISON ---
TCP: Reliable but slower
UDP: Faster but may lose packets
```

21.) Capture the FTP username and password using wireshark.

Ans:- To capture and analyze FTP traffic using Wireshark and observe FTP authentication details (username and password) in a controlled network environment.

ALGORITHM

- 1. Install and open Wireshark**
- 2. Select active network interface (Ethernet/Wi-Fi)**
- 3. Start packet capture**
- 4. Connect to a local FTP server (e.g., FileZilla Server / localhost FTP)**
- 5. Login using FTP client**
- 6. Stop capture in Wireshark**
- 7. Apply filter:**

ftp || ftp-data

- 8. Inspect packets for USER and PASS commands**
- 9. Note username and password (visible in plain FTP)**

PROGRAM/PROCEDURE :-

Step 1: Start Wireshark

- Open Wireshark**
 - Select network interface**
 - Click Start Capture**
-

Step 2: Start FTP Server (Local)

- **Use FileZilla Server or any FTP server**
 - **Create a test user:**
 - **Username: testuser**
 - **Password: test123**
-

Step 3: Connect via FTP Client

- **Use command prompt or FileZilla client:**

ftp localhost

Login:

Username: testuser

Password: test123

Step 4: Stop Capture

- **Go back to Wireshark**
 - **Click Stop Capture**
-

Step 5: Apply Filter

ftp

OUTPUT:-

In Wireshark packets:

USER testuser

PASS test123

These appear in plain text because FTP is not encrypted.

23.) Write a program for a HDLC frame to perform the following:

- . Bit stuffing**
- . Character stuffing**

Ans:- AIM

To implement HDLC framing techniques using:

- 1. Bit Stuffing**
 - 2. Character Stuffing**
-

□ ALGORITHM

◆ Bit Stuffing

1. Take input binary data
 2. Scan bits
 3. After every 5 consecutive 1s, insert a 0
 4. Display stuffed data
-

◆ Character Stuffing

1. Take input frame data
2. Define flag character (e.g., F)
3. Define escape character (e.g., E)
4. If flag/escape appears in data, add escape before it
5. Display stuffed frame

PROGRAM:-

```
import java.util.*;

public class HDLCStuffing {

    // ----- BIT STUFFING -----
    static void bitStuffing(String data) {
        StringBuilder result = new StringBuilder();
        int count = 0;

        for (int i = 0; i < data.length(); i++) {
            char bit = data.charAt(i);
            result.append(bit);

            if (bit == '1') {
                count++;
            } else {
                count = 0;
            }

            // after 5 consecutive 1s, insert 0
            if (count == 5) {
                result.append('0');
                count = 0;
            }
        }

        System.out.println("\nBit Stuffed Data: " + result);
    }

    // ----- CHARACTER STUFFING -----
    static void charStuffing(String data) {
        String flag = "F";
        String escape = "E";

        StringBuilder result = new StringBuilder();

        result.append(flag); // start flag

        for (int i = 0; i < data.length(); i++) {
            String ch = String.valueOf(data.charAt(i));

            if (ch.equals(flag) || ch.equals(escape)) {
                result.append(escape);
            }

            result.append(ch);
        }

        result.append(flag); // end flag

        System.out.println("Character Stuffed Frame: " + result);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter binary data (Bit Stuffing): ");
        String bitData = sc.nextLine();
    }
}
```

```

        System.out.print("Enter character data (Char Stuffing): ");
        String charData = sc.nextLine();

        bitStuffing(bitData);
        charStuffing(charData);

        sc.close();
    }
}

```

OUTPUT:-

```

Enter binary data (Bit Stuffing): 1111101111110
Enter character data (Char Stuffing): AFBECF

Bit Stuffed Data: 111110011111010
Character Stuffed Frame: FAEFBEECEFF

```

24.) a) To print the day and time that connect store a string from the server and displays the string to the client.

Ans:- AIM

To implement a TCP client-server program where the server sends the current date

and time as a string and the client displays it.

□ ALGORITHM

Server:

- 1. Create ServerSocket**
- 2. Accept client connection**
- 3. Get current date & time**
- 4. Send string to client**
- 5. Close connection**

Client:

- 1. Create socket connection**
- 2. Receive string from server**
- 3. Display message**
- 4. Close connection**

PROGRAM:-

SERVER:-

```
import java.io.*;
import java.net.*;
import java.time.*;

public class DateTimeServer {
    public static void main(String[] args) {
        try {
            ServerSocket ss = new ServerSocket(5000);
            System.out.println("Server started...");

            Socket s = ss.accept();
            System.out.println("Client connected");

            PrintWriter out = new PrintWriter(s.getOutputStream(), true);

            String dateTime = LocalDateTime.now().toString();

            out.println("Current Date & Time: " + dateTime);

            s.close();
            ss.close();

        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

CLIENT:-

```
import java.io.*;
import java.net.*;

public class DateTimeClient {
    public static void main(String[] args) {
        try {
            Socket s = new Socket("localhost", 5000);

            BufferedReader in = new BufferedReader(
                new InputStreamReader(s.getInputStream()));

            String msg = in.readLine();

            System.out.println("Message from server:");
            System.out.println(msg);

            s.close();
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

OUTPUT:-

SERVER:-

```
Server started...
Client connected
```

CLIENT:-

```
Message from server:
Current Date & Time: 2026-05-06T23:33:03.121010
```

25.) Analyze the TCP performance using simulation tool.

Ans:- AIM

To analyze TCP performance using Java simulation by measuring packet delivery, delay, and reliability.

ALGORITHM

- 1. Generate packets**
- 2. Send packets using TCP simulation**
- 3. Assume ACK for every packet**
- 4. Add delay for each transmission**
- 5. Simulate congestion loss (rare)**
- 6. Compute:**
 - delivered packets**

- total delay
- throughput

PROGRAM :-

```
import java.util.*;

public class TCPSimulation {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Random rand = new Random();

        System.out.print("Enter number of packets: ");
        int packets = sc.nextInt();

        int delivered = 0;
        int lost = 0;
        long totalDelay = 0;

        System.out.println("\n--- TCP Transmission Simulation ---\n");

        for (int i = 1; i <= packets; i++) {

            int delay = rand.nextInt(50) + 50; // 50-100 ms

            try {
                Thread.sleep(delay);
            } catch (Exception e) {}

            // simulate rare packet loss (TCP retransmission)
            int chance = rand.nextInt(100);

            if (chance < 90) { // 90% success
                System.out.println("Packet " + i + " delivered (ACK received)");
                delivered++;
                totalDelay += delay;
            } else {
                System.out.println("Packet " + i + " lost → retransmitting...");
                lost++;

                // retransmission
                System.out.println("Packet " + i + " retransmitted and delivered");
                delivered++;
                totalDelay += delay * 2;
            }
        }

        double throughput = (double) delivered / (totalDelay / 1000.0);

        System.out.println("\n--- TCP PERFORMANCE RESULT ---");
        System.out.println("Total Packets: " + packets);
        System.out.println("Delivered: " + delivered);
        System.out.println("Lost: " + lost);
        System.out.println("Total Delay: " + totalDelay + " ms");
        System.out.println("Throughput: " + throughput + " packets/sec");

        sc.close();
    }
}
```

OUTPUT:-

```
Enter number of packets: 5
```

```
--- TCP Transmission Simulation ---
```

```
Packet 1 lost ? retransmitting...
```

```
Packet 1 retransmitted and delivered
```

```
Packet 2 delivered (ACK received)
```

```
Packet 3 delivered (ACK received)
```

```
Packet 4 delivered (ACK received)
```

```
Packet 5 delivered (ACK received)
```

```
--- TCP PERFORMANCE RESULT ---
```

```
Total Packets: 5
```

```
Delivered: 5
```

```
Lost: 1
```

```
Total Delay: 400 ms
```

```
Throughput: 12.5 packets/sec
```